INTRODUCCIÓN A JAVA (III)

Programación

FUNCIONES

Las **FUNCIONES** (**subprogramas**) son **utilizadas para evitar la repetición de código** en un programa al poder ejecutarlo desde varios puntos de un programa con sólo invocarlo.

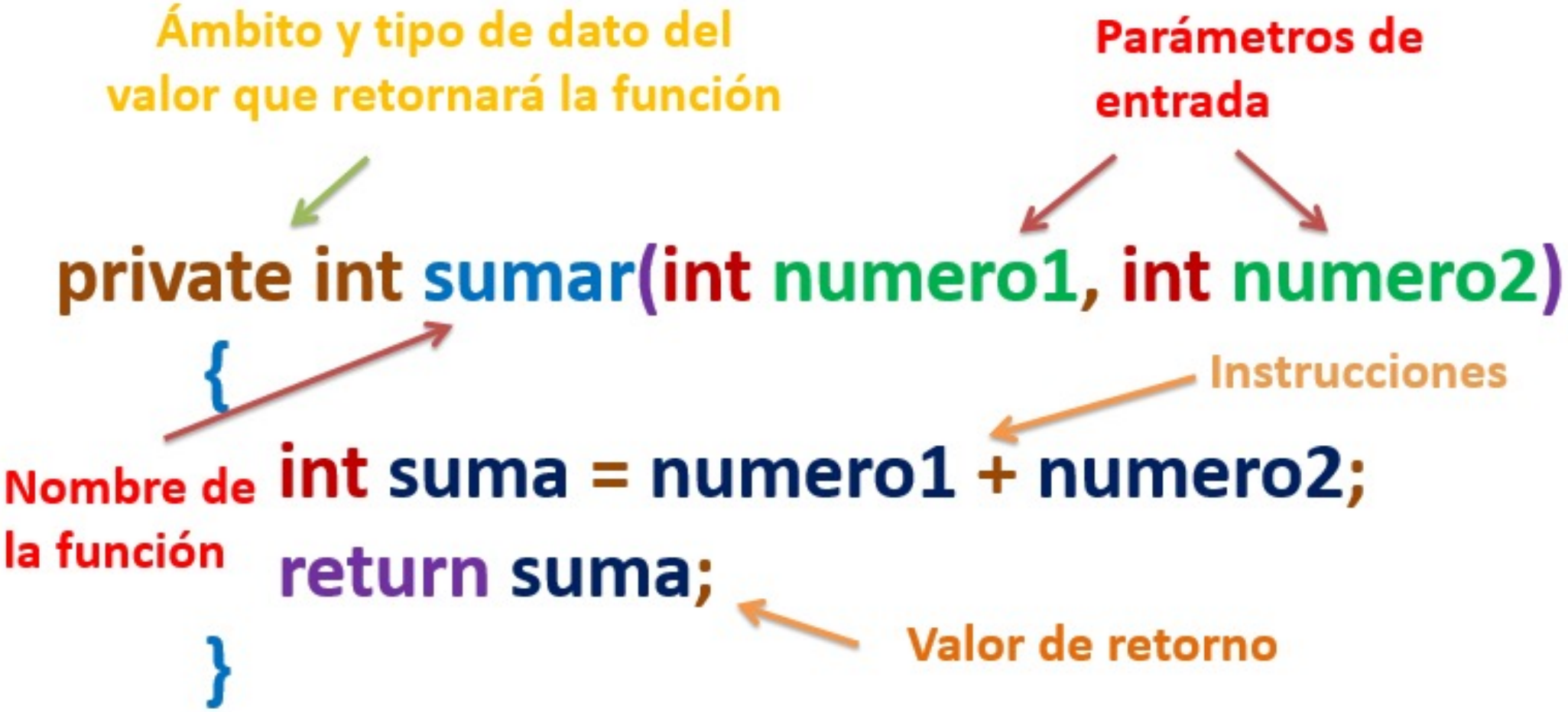
Utilidad principal:

- Agrupar código que forma una idea concreta.
- Agrupar código que se necesitará varias veces en un programa.
- Dividir el código de un programa grande un subprogramas (funciones), cada uno de ellos especializados en resolver una parte del problema.

Características:

- Se definen mediante un nombre único que representa el bloque de código.
- Pueden ser llamadas (ejecutadas) desde cualquier parte del código.
- Se les puede pasar valores para que los procesen de alguna forma.
- Pueden devolver un resultado para ser usado desde donde se les llamó.

DECLARACIÓN DE UNA FUNCIÓN



EJEMPLOS

```
public static void imprimeHolaMundo() {
        System.out.println(“Hola mundo”);
}

public static void imprimeHolaNombre(String nombre) {
        System.out.println(“Hola ” + nombre);
}
```

EJEMPLOS

```
public static int doble(int a) {
        int resultado = a * 2;
        return resultado;
}

public static int multiplica(int a, int b) {
        int resultado = a * b;
        return resultado;
}
```

EJEMPLOS

```
public static double maximo(double valor1, double valor2) {
        double max;
        if (valor1 > valor2)
                max = valor1;
        else
                max = valor2;
        return max;
}
```

LLAMADA A UNA FUNCIÓN

Las funciones pueden ser invocadas o llamadas desde cualquier otra función, incluida ella misma.

```
public static void main(String[] args) {
        imprimeHolaMundo();

        int a = doble(10);
        int b = multiplica(3, 5);

        int c = doble(a);
        int d = multiplica(a, b);

        int e = doble(4) + multiplica(2,10);
        System.out.println("El doble de 35 es "
                + doble(35) );
        System.out.println("12 por 12 es " +
                multiplica(12,12) );
}
```

FUNCIONES, PROCEDIMIENTOS y MÉTODOS

FUNCIONES

Conjunto de instrucciones, encapsulados en un bloque, usualmente reciben parámetros, cuyos valores utilizan para efectuar operaciones y adicionalmente **retornan un valor con la ejecución return.**

PROCEDIMIENTOS

Conjunto de instrucciones que **se ejecutan sin retornar ningún valor.**

MÉTODOS

Está asociado a un objeto, SIEMPRE, básicamente un método es **una función o procedimiento que pertenece a un objeto o clase.**

PARÁMETROS

PASO POR VALOR

Parámetros **DE TIPO SIMPLE c**omo int, double, boolean, char, etc.

El valor se copia al parámetro y por lo tanto si se modifica dentro de la función esto no afectará al valor fuera de ella porque **son variables distintas**.

PASO POR REFERENCIAS

Parámetros **DE TIPO OBJETO c**omo objetos de tipo String, los Arrays, etc.

No se copia el objeto sino que se le pasa a la función una referencia al objeto original (un puntero).

Desde la función se accede directamente al objeto que se encuentra fuera.

Los cambios que hagamos dentro de la función afectarán al objeto.

EXCEPCIONES

Una **EXCEPCIÓN** es un error semántico que se produce en tiempo de ejecución.

Aunque un código sea correcto sintácticamente, es posible que durante su ejecución se produzcan errores inesperados, como por ejemplo:

- Dividir por cero.
- Intentar acceder a una posición de un array fuera de sus límites.
- Al llamar al nextInt() de un Scanner el usuario no introduce un valor entero.
- Intentar acceder a un fichero que no existe o que está en un disco duro corrupto.
- Etc.

EJEMPLO

```
12  public class Ejemplos_excepciones {
13
14      public static void main(String[] args) {
15          int div, x, y;
16
17          x = 3;
18          y = 0;
19
20          div = x / y;
21
22          System.out.println("El resultado es " + div);
23      }
24
25  }
```

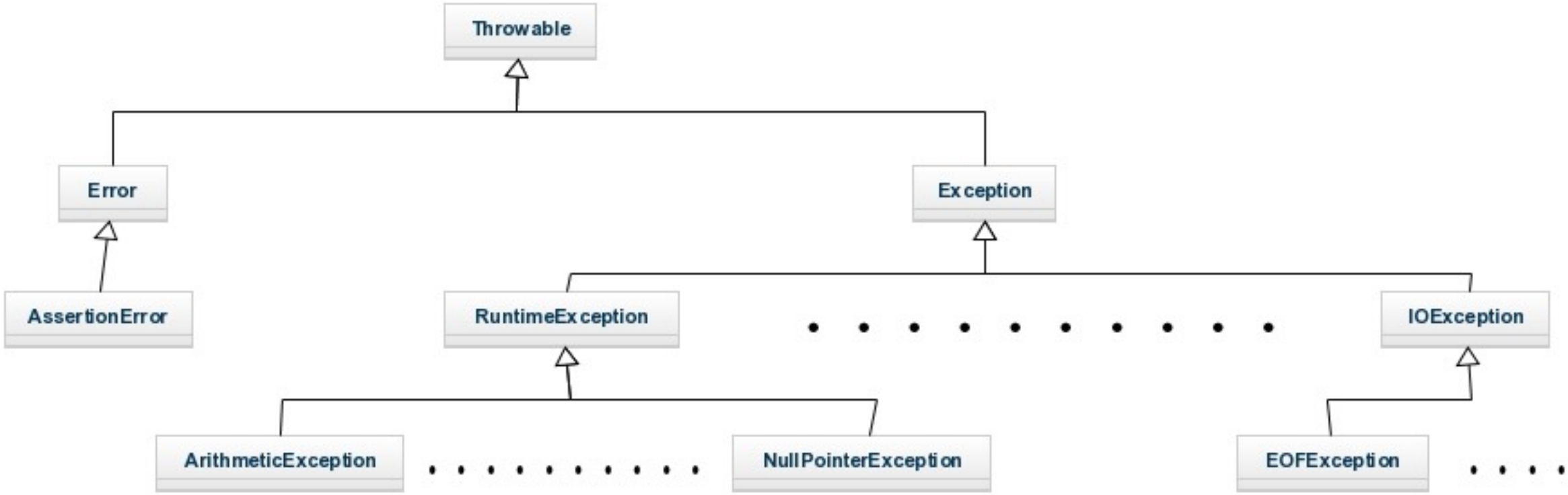
```
run:
Exception in thread "main" java.lang.ArithmeticException: / by zero
        at ejemplos_excepciones.Ejemplos_excepciones.main(Ejemplos_excepciones.java:20)
C:\Users\user\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)
```

EJEMPLO

```
12    public class Ejemplos_excepciones {
13
14        public static void main(String[] args) {
15            String cadena = "56s";
16            int num;
17
18            num = Integer.parseInt(cadena);
19
20            System.out.println("El número es " + num);
21        }
```

```
run:
Exception in thread "main" java.lang.NumberFormatException: For input string: "56s"
        at java.lang.NumberFormatException.forInputString(NumberFormatException.java:65)
        at java.lang.Integer.parseInt(Integer.java:580)
        at java.lang.Integer.parseInt(Integer.java:615)
        at ejemplos_excepciones.Ejemplos_excepciones.main(Ejemplos_excepciones.java:18)
C:\Users\user\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1
BUILD FAILED (total time: 0 seconds)
```

JERARQUÍA Y TIPOS DE EXCEPCIONES JAVA



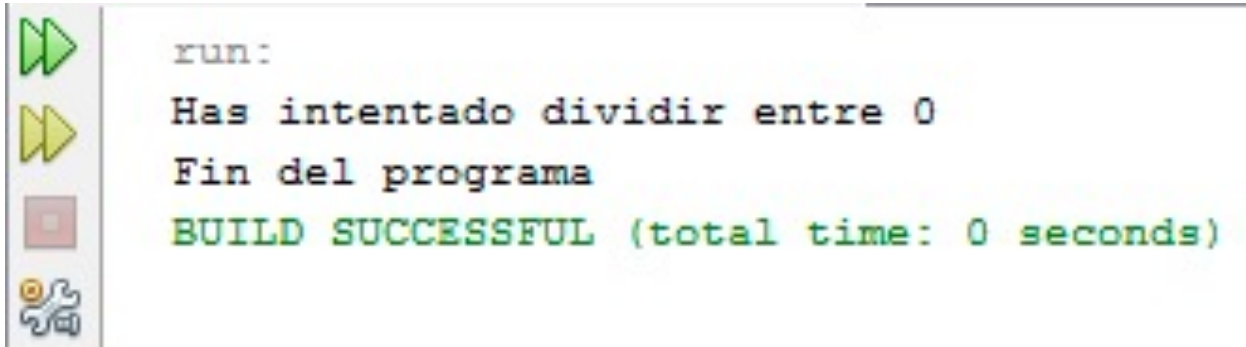
MANEJADOR DE EXCEPCIONES

Es una bloque de código encargado de tratar las excepciones para intentar recuperarse del fallo y **evitar que la excepción sea lanzada descontroladamente hasta el main y termine el programa.**

```
try {
        // Instrucciones que podrían
        lanzar una excepción.
}
catch (TipoExcepción nombreVariable)
{
        // Instrucciones que se
        ejecutan cuando 'try' lanza una
        excepción.
}
finally {
        // Instrucciones que se
        ejecutan tanto si hay excepción
        como si no.
}
```

EJEMPLO

```
12  public class Ejemplos_excepciones {
13
14      public static void main(String[] args) {
15          int x = 1, y = 0;
16
17          try
18          {
19              int div = x / y;
20
21              System.out.println("La ejecución no llegará aquí.");
22          }
23          catch(ArithmeticException ex)
24          {
25              System.out.println("Has intentado dividir entre 0");
26          }
27
28          System.out.println("Fin del programa");
29      }
30  }
```



CLÁUSULA CATCH

Es importante entender que el bloque catch solo capturará excepciones del tipo indicado.

```
try {// instrucciones que pueden producir distintos tipos de Excepciones

}

catch (TipoExcepción1 e1) {// instrucciones para manejar un TipoExcepción1

}

catch (TipoExcepción2 e2) {// instrucciones para manejar un TipoExcepción2

}

catch (TipoExcepciónN eN) {// instrucciones para manejar un TipoExcepciónN

}

finally { // opcional// instrucciones que se ejecutarán tanto si hay excepción como si no

}
```

EJEMPLO

```
14  public class Ejemplos_excepciones {
15
16      public static void main(String[] args) {
17          int x, y, div, pos;
18          int[] v = {1,2,3};
19          Scanner in = new Scanner(System.in);
20
21          try
22          {
23              System.out.print("Introduce el numerador: ");
24              x = in.nextInt();
25
26              System.out.print("Introduce el denominador: ");
27              y = in.nextInt();
28
29              div = x / y;
30
31              System.out.println("La división es " + div );
32
33              System.out.print("Introduce la posición del vector a consultar: ");
34              pos = in.nextInt();
35
36              System.out.println("El elemento es " + v[pos] );
37
38          }
```

EJEMPLO

```
39              catch(ArithmeticException ex)
40              {
41                  System.out.println("División por cero: " + ex);
42              }
43              catch(ArrayIndexOutOfBoundsException ex)
44              {
45                  System.out.println("Sobrepasado el tamaño del vector: " + ex);
46              }
47
48              System.out.println("Fin del programa");
49          }
50      }
```

EL OBJETO EXCEPTION

Toda excepción genera un objeto de la clase Exception (o uno más específico que hereda de Exception). Dicho objeto contendrá detalles sobre el error producido.

Los dos métodos de Exception más útiles son:

- **getMessage()** → Devuelve un String con un texto simple sobre el error.

- **printStackTrace()** → Es el que más información proporciona. Indica qué tipo de Excepción se ha producido, el mensaje simple, y también toda la pila de llamadas. Esto es lo que hace Java por defecto cuando una excepción no se maneja y acaba parando el programa.

LANZAR EXCEPCIONES

Un programador puede programar su código de forma que se lancen excepciones cuando se intente hacer algo incorrecto o inesperado.

Palabra reservada **throw** seguido de un objeto de tipo Exception.

```
throw new Exception();
```

Esto es equivalente a :

```
Exception e = new Exception();
throw e;
```

El constructor ¡permite un argumento String para dar detalles sobre el problema.

```
throw new Exception("La edad no
        puede ser negativa");
```

INDICAR LA EXCEPCIÓN EN LA CABECERA DEL MÉTODO

```
public void setEdad(int edad) throws Exception {
        if (edad < 0)
                throw new Exception("Edad negativa no permitida");
        else
                this.edad = edad;
        }
}
```

DEFINIR EXCEPCIONES PROPIAS

```
14    public class Ejemplos_excepciones {
15
16        public static void main(String[] args) {
17            try
18            {
19                metodo(1);
20                metodo(20);
21            }
22            catch (ExcepcionPropia e)
23            {
24                System.out.println("capturada :" + e);
25            }
26        }
```

DEFINIR EXCEPCIONES PROPIAS

```
28          static void metodo(int n) throws ExcepcionPropia
29          {
30              System.out.println("Llamado por metodo(" + n +")");
31
32              if (n > 10)
33                  throw new ExcepcionPropia(n);
34
35              System.out.println("Finalización normal");
36          }
37      }
```

DEFINIR EXCEPCIONES PROPIAS

```
12    public class ExcepcionPropia  extends Exception
13    {
          private int num;
15
16        ExcepcionPropia(int n)
17        {
18            this.num = n;
19        }
          public String toString()
21        {
22            return "Excepcion Propia[" + this.num + "]";
23        }
24
25    }
```